

Краткая инструкция по первому использованию Gnu Radio

Введение	1
Установка на Windows.....	1
Установка на Linux.....	1
Использование	2
Начало работы	2
Основные понятия.....	5
Первый граф	12
Создание блоков.....	15
Расширение GNU Radio	17

Введение

GNU Radio — это свободная программная платформа для разработки радиоприложений. Это инструмент, который позволяет вам взять блоки радиоприложений и соединить их, чтобы создать собственное радио.

GNU Radio предоставляет набор блоков обработки сигналов, которые можно использовать для создания программного радио. Эти блоки включают в себя все, от простых математических операций, таких как сложение и умножение, до сложных алгоритмов, таких как модуляция и демодуляция, кодирование и декодирование, фильтрация и многое другое.

GNU Radio поддерживает множество аппаратных платформ SDR (Software-Defined Radio), что делает его универсальным инструментом для исследования и разработки радиоприложений.

Установка на Windows

Для того чтобы установить Gnuradio на Windows необходимо скачать дистрибутив с официального сайта:

https://wiki.gnuradio.org/index.php?title=InstallingGR#Quick_Start

После чего установить, следуя инструкциям, после этого приложение можно будет найти в поиске.

Установка на Linux

1. Откройте терминал.
2. Установите GNURadio, используя менеджер пакетов APT. Для этого введите следующие команды:

```
sudo apt-get install gnuradio
```

3. После установки вы можете запустить GNURadio, введя в терминале: `gnuradio-companion`.

Можно установить Gnu Radio сразу с предустановленными блоками

Инструкция тут: <https://github.com/ryanvolz/radioconda>

Использование

Команды в Linux вводятся в терминале, для вызова терминала в Linux нажмите сочетание клавиш **Ctrl+Alt+T**, или правой кнопкой мыши на свободном месте экрана и выбрать **Open in Terminal**.

Начало работы

GNU Radio Companion (GRC) — это графический интерфейс для проектирования радиоприложений. Вот как его можно использовать:

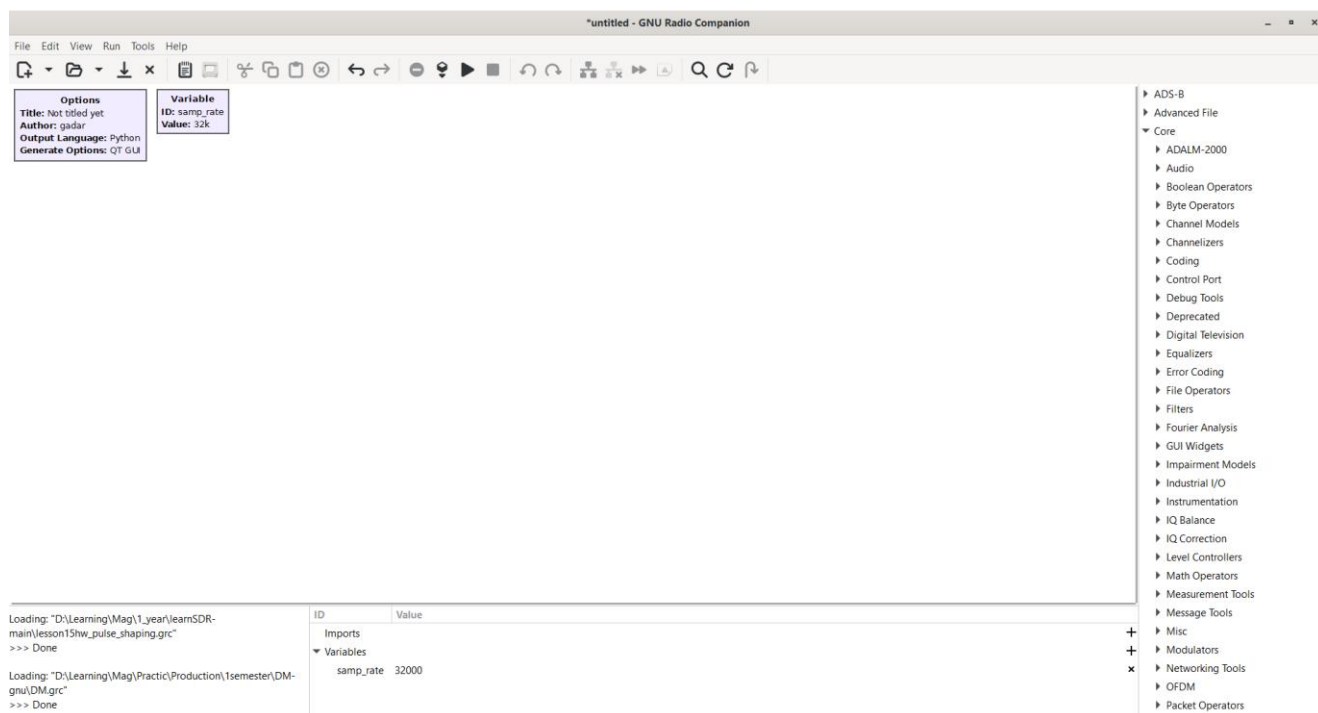
- **Запуск GRC**

На системах Linux GRC вызывается командой

```
gnuradio-companion
```

Если установка прошла успешно, GRC откроется в своем собственном окне.

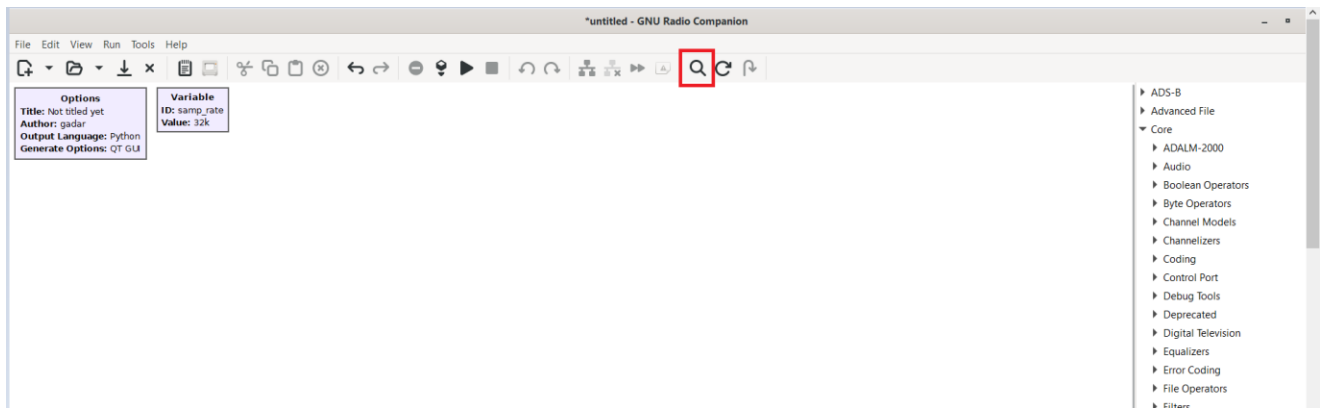
В системах Windows в меню пуск должно появиться приложение GNU Radio Companion.



- **Поиск блоков**

Библиотека содержит различные блоки, установленные в путях блоков GRC. Здесь мы находим блоки, которые предустановлены в GNU Radio, и блоки, которые установлены в системе. Если вы хотите найти блок, но не знаете, в какой категории его искать, вы можете использовать функцию поиска, нажав на значок лупы или

набрав Ctrl+f, а затем начать вводить ключевое слово для идентификации блока.

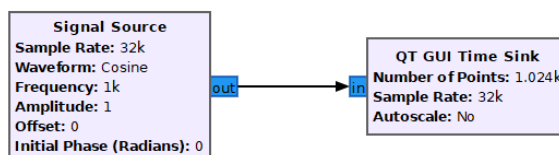
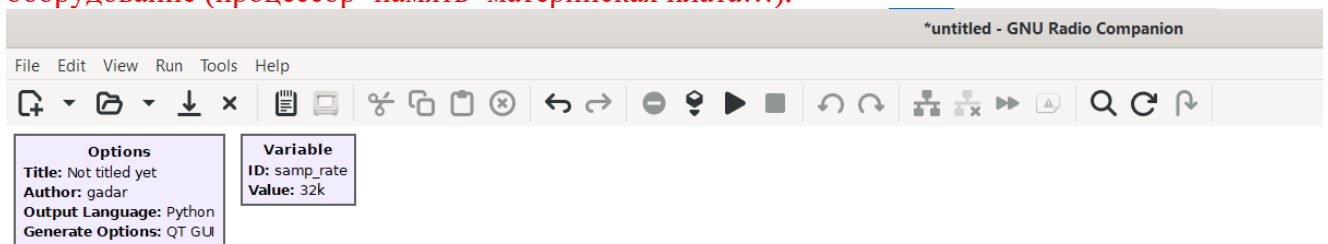


- **Создание графа потока:** Вы можете перетаскивать блоки из библиотеки в рабочее пространство и соединять их, кликая по их портам.

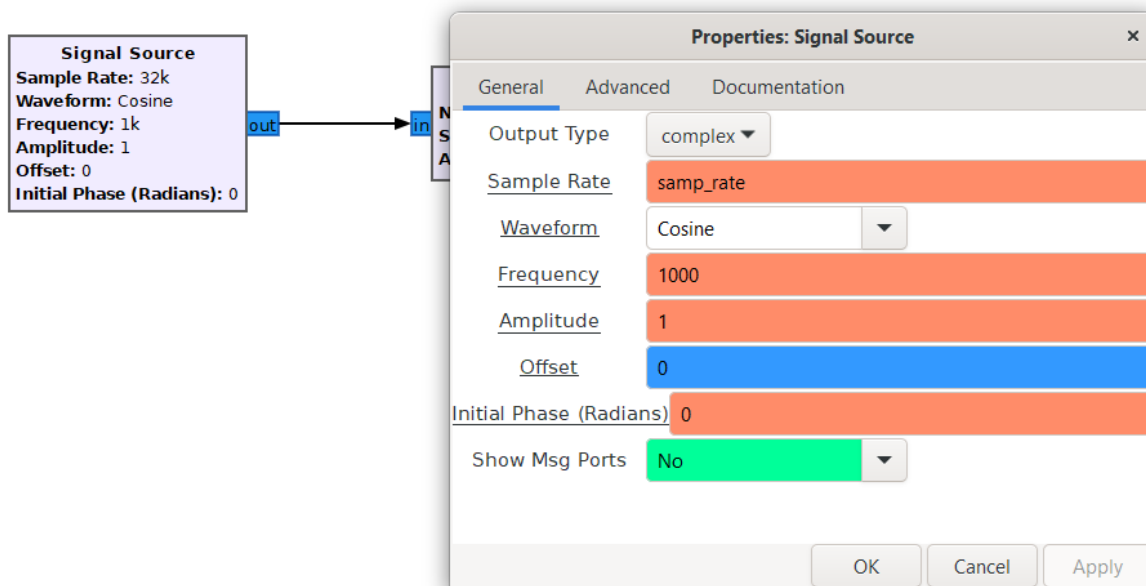
ВНИМАНИЕ! Здесь и далее в инструкции допущена ошибка: отсутствует блок Throttle – блок, который используется всегда, если не используется аппаратные источники или получатели данных (без подключения SDR приёмника или источника аудио сигналов ДОЛЖЕН использоваться ОБЯЗАТЕЛЬНО)



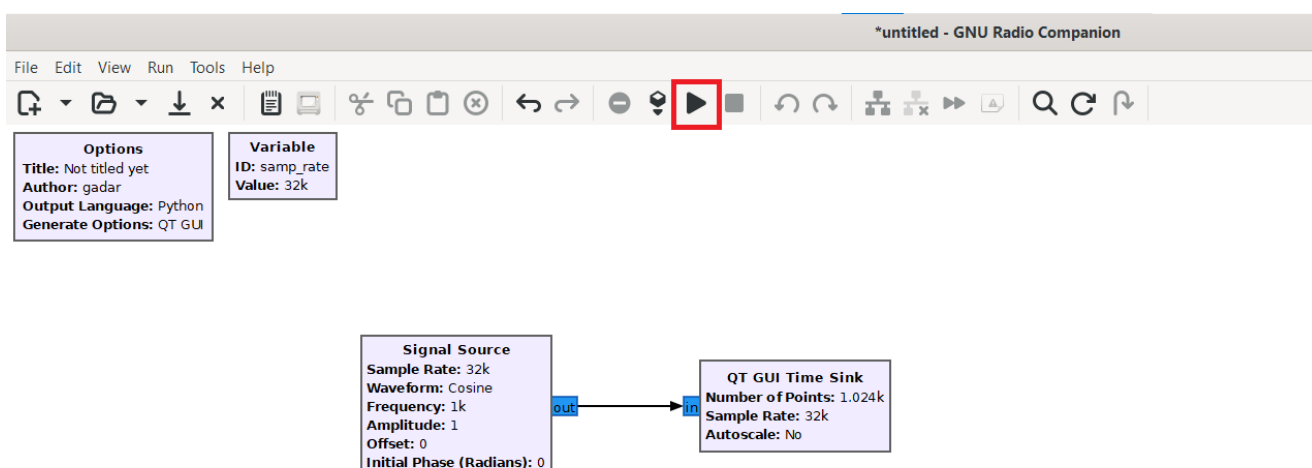
Если его не использовать, обработка сигналов будет не на указанной Вами тактовой частоте (samp_rate), а на максимальной частоте, которую позволяет сделать Ваше оборудование (процессор+память+материнская плата...).



- **Изменение свойств блоков:** Рабочее пространство (основная область экрана) содержит все наши блоки, которые составляют наш граф потока, и внутри каждого блока мы можем видеть все различные параметры блока.



- **Сохранение и запуск графа потока**
После того, как вы создали граф потока, вы можете сохранить его и запустить, нажав на кнопку “Выполнить” (она выглядит как треугольник, указывающий вправо) на панели инструментов.



Как только мы запустим граф мы увидим, что в папке с нашим графом появился python файл. Именно он является исполняемым файлом. Гнурадио лишь генерирует python файл и запускает его, если мы просто запустим этот файл через консоль, то увидим, что программа так же выполняется.

Основные понятия

Создадим наш первый простейший граф. Мы сразу замечаем, что наше рабочее пространство не пустое, в левом верхнем углу мы видим 2 блока.

Options	Variable
Title: Not titled yet	ID: samp_rate
Author: gadar	Value: 32k
Output Language: Python	
Generate Options: QT GUI	

Первый блок является основой нашего графа, в нем указываются все основные параметры.

Properties: Options

General | Advanced | Documentation

ID: TEST

Title: Not titled yet

Author: gadar

Copyright:

Description:

Output Language: Python

Generate Options: QT GUI

Run: Autostart

Max Number of Output: 0

Realtime Scheduling: Off

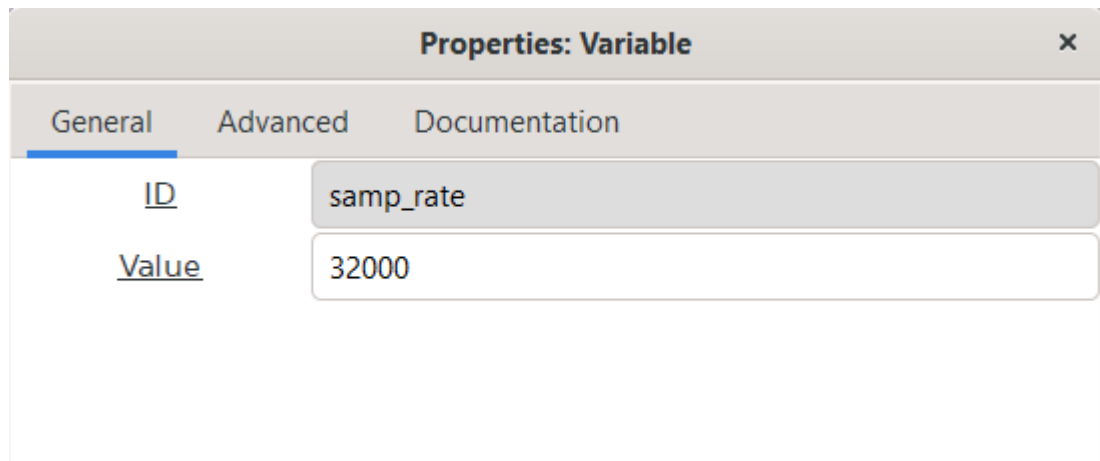
QSS Theme:

OK Cancel Apply

Первый параметр ID принимает значение названия нашего графа. Он отвечает за название генерируемого python файла. Следующие 4 параметра ни на что не влияют, они просто обозначают заголовок, автор, авторские права и описание графа. Эти параметры помогут разобраться в программе другому человеку, который решит воспользоваться вашим графом. Можете подробнее ознакомиться с этим блоком по ссылке:

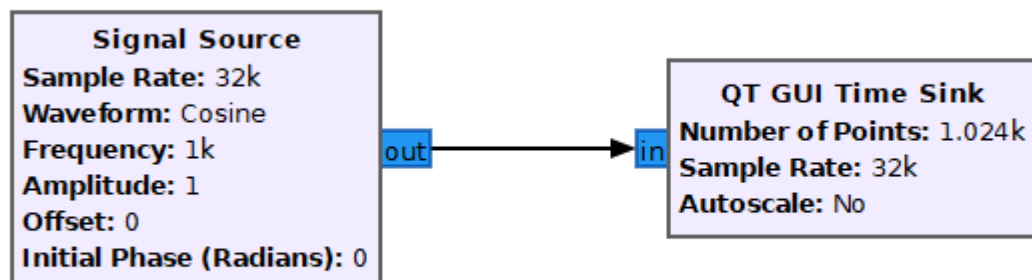
<https://wiki.gnuradio.org/index.php/Options>.

Второй блок является переменной, которую мы задаем.

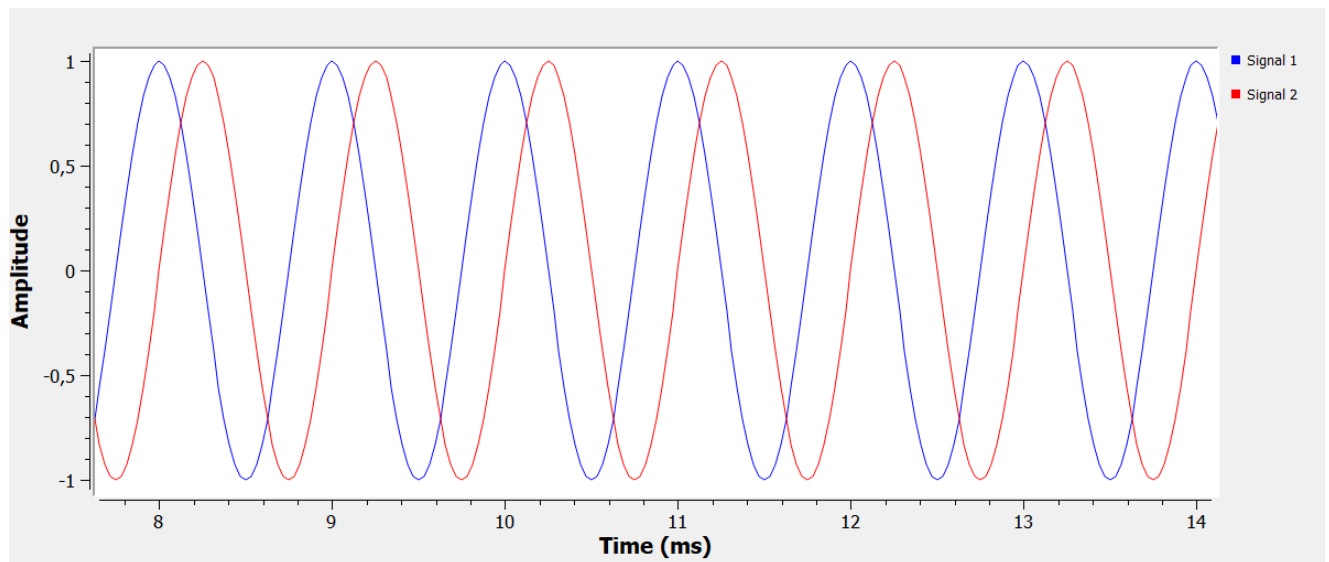


Параметр ID – это имя переменной, Value – значение. После этого мы можем использовать эту переменную в остальных блоках этого графа. Переменная `samp_rate` является частотой дискретизации, и задаётся по умолчанию со значением 32000, потому что почти все блоки гнурадио будут использовать эту переменную, и в их параметрах по умолчанию в графе Sample Rate стоит переменная `samp_rate`.

Найдите блок Signal Source и QT GUI Time Sink. Первый блок является генератором сигнала, и любые блоки, в которых есть слово Source является генератором какого-либо сигнала. Второй блок строит нам график этого сигнала от времени.



Все значения оставляем по умолчанию и запускаем программу. И видим комплексную косинусоиду.



Теперь разберемся почему, если мы откроем параметры нашего генератора, то увидим следующую картину

General Advanced Documentation

Output Type

Sample Rate

Waveform

Frequency

Amplitude

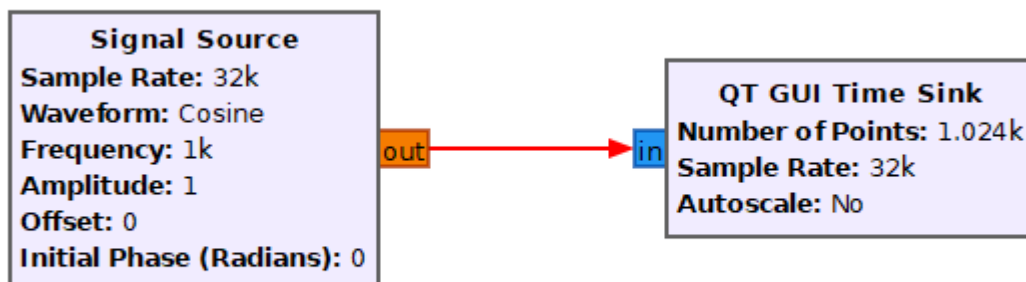
Offset

Initial Phase (Radians)

Show Msg Ports

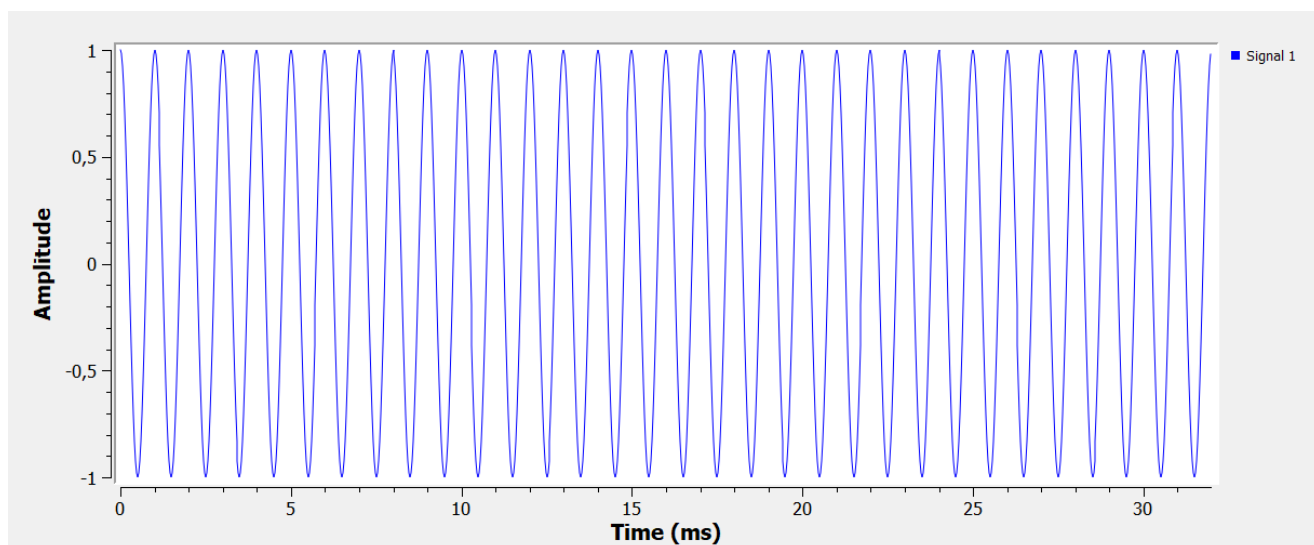
OK Cancel Apply

Самый первый параметр, это тип выходных данных, попробуем поменять его на float. Не забудьте нажать Apply и закрыть окно, или же можно просто нажать ОК, эта кнопка аналогична прошлому действию.

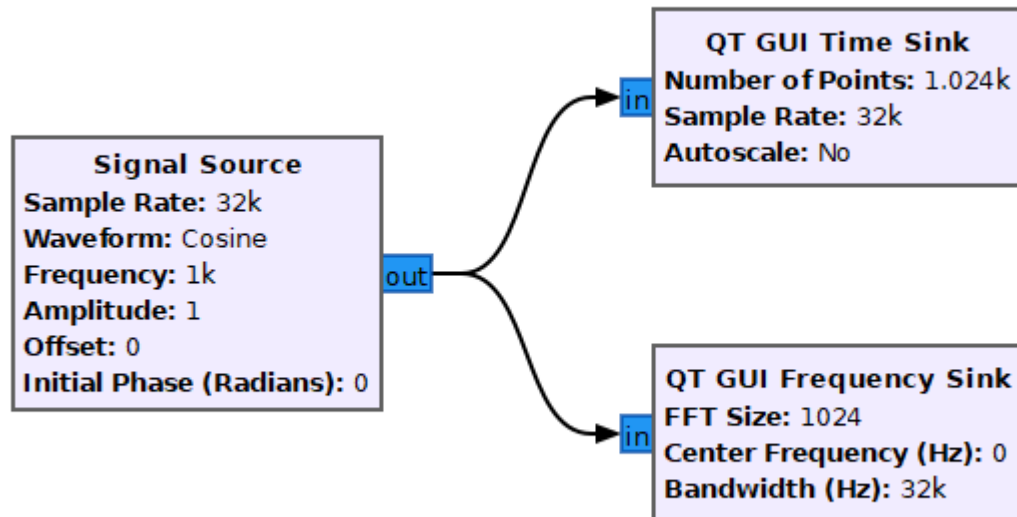


Мы видим, что цвет выходных параметров поменялся, и стрелочка горит красным цветом и не позволяет запустить наш граф. Это происходит, потому что мы пытаемся подать в блок, который принимает данные вида complex, поток формата float. Чтобы исправить эту ошибку необходимо зайти в параметры блока QT GUI Time Sink, и изменить первый параметр Type так же на float. Так же это можно сделать быстрее, нажмите на блок, который хотите изменить, его рамка станет из черного бирюзовой, после этого нажмите стрелочку вверх или вниз. Так можно быстро менять формат данных, которые подключаются к портам. Если нажать стрелочки влево или вправо, то блок перевернется, это не влияет ни на что кроме внешнего вида.

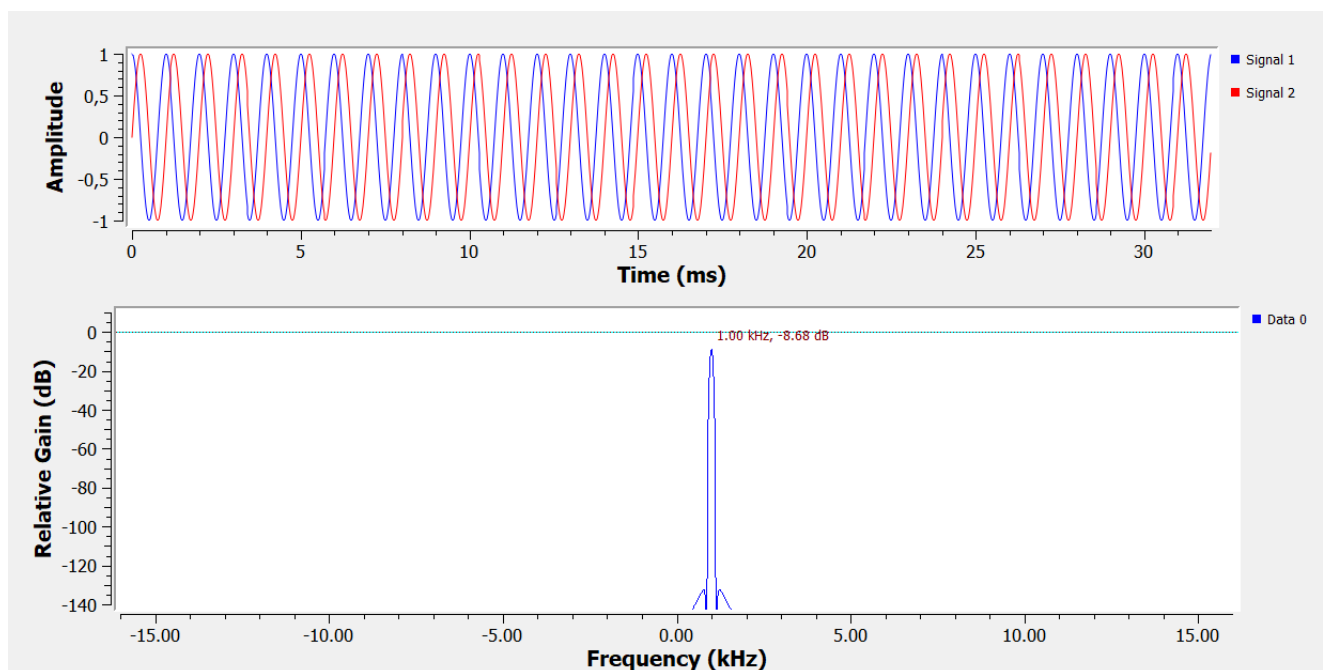
После запуска мы видим такую картину



Как видим, теперь нет второго графика, который изображает мнимую часть сигнала. Вернём обратно комплексный формат данных и добавим ещё один блок, который будет нам показывать спектр сигнала QT GUI Frequency Sink.

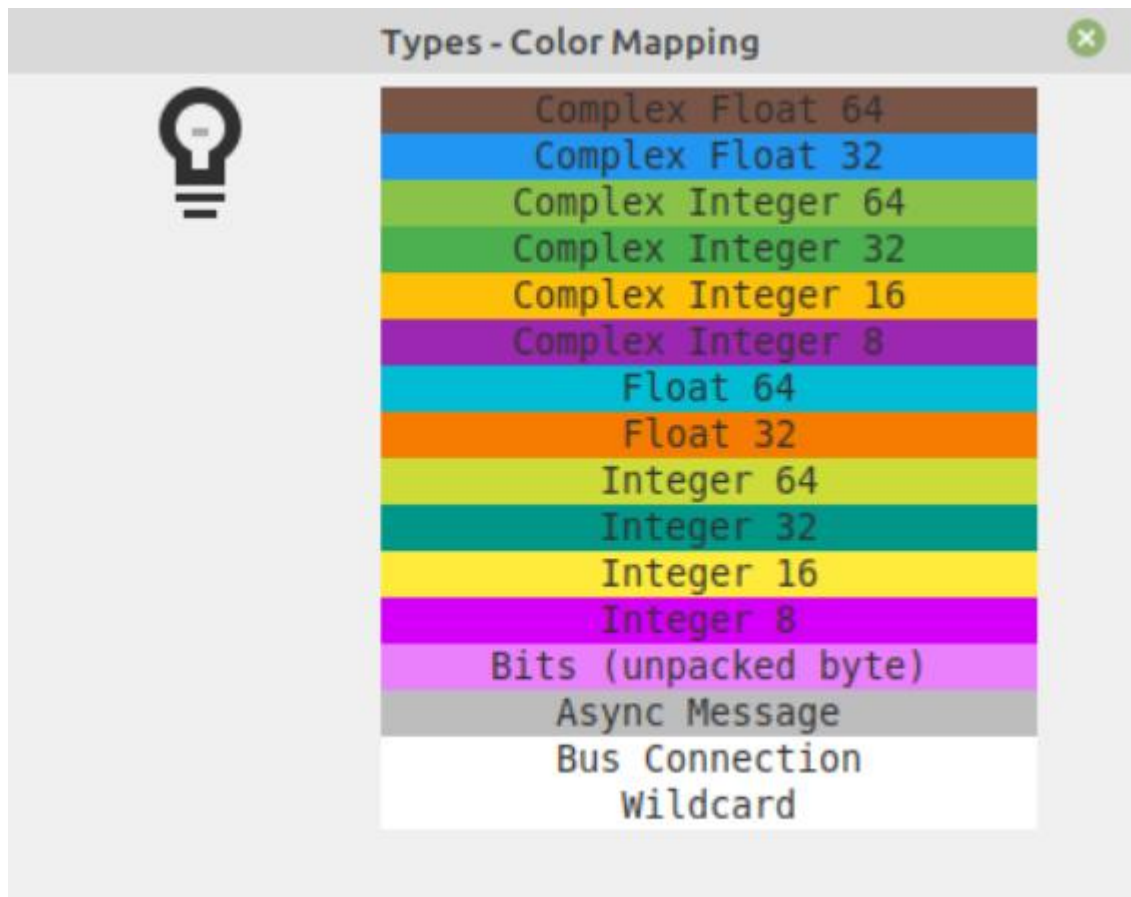


После запуска увидим следующую картину

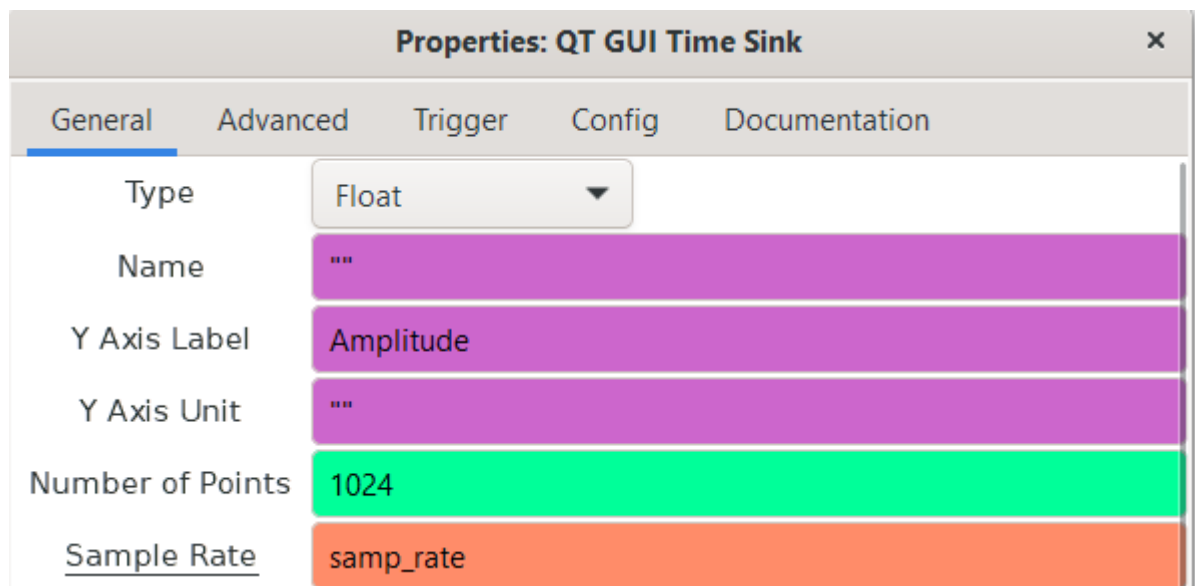


Теперь мы видим графическое отображение частоты. Если измените значение частоты на источнике, то увидите, что пик сместился на эту частоту. Если вы укажете слишком большую частоту источника, а именно больше 16000, то спектр будет показывать неверные значения частоты. Это происходит из-за того, что частота дискретизации является слишком маленькой для того, чтобы правильно обработать такой сигнал, чтобы он правильно указывался на спектре `samp_rate` должен быть в 2 раза больше, чем частота сигнала. Заметьте, что после изменения частоты дискретизации так же расширились границы спектра. Вы можете попробовать изменить и остальные параметры, посмотреть, на что они влияют.

Вспомним, что при смене типа данных сменился цвет порта. Каждый цвет обозначает свой тип данных:



Также вы, возможно, заметили, что параметры внутри блоков тоже имеют разные цвета, например мы видим в блоке QT GUI Time Sink 3 цвета, не считая белого. Это значит, что эти параметры должны быть определенного типа.

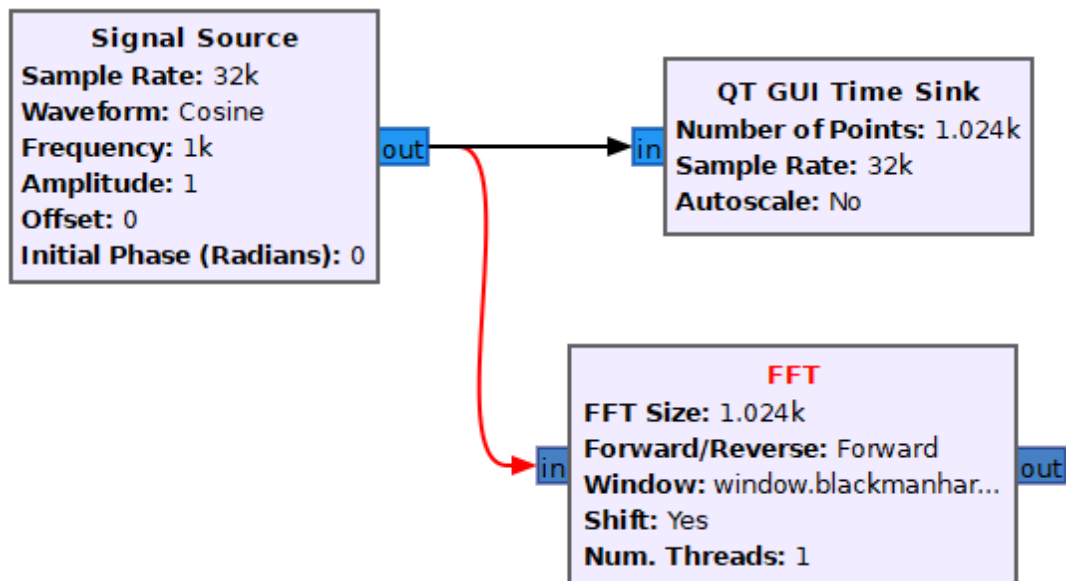


Всего есть 4 цвета:

- Оранжевый – float (число с плавающей запятой)
- Зеленый – int (целое число)
- Синий – complex (комплексное число)

- Фиолетовый – string (строка текста)

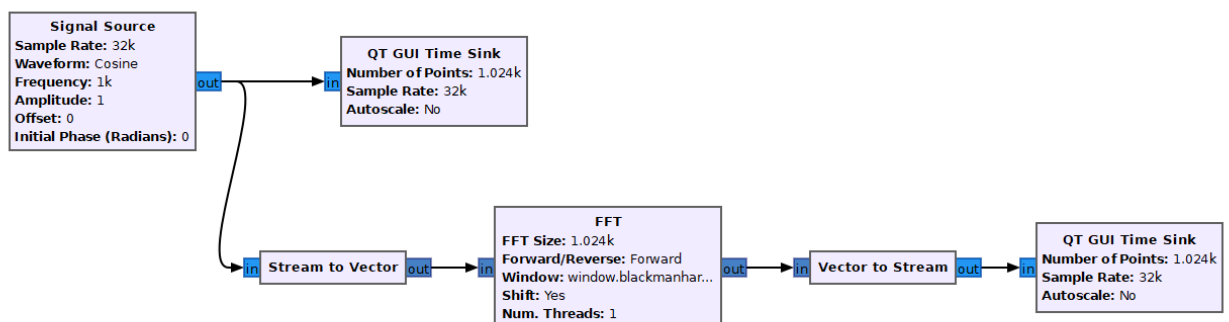
Теперь добавим новый блок FFT



Мы видим, что у этого блока входной порт тоже синего цвета, но чуть темнее, и в таблице такого нет. И с другим синим цветом не соединяется. Дело в том, что блок Signal Source выводит поток комплексных чисел, а блок FFT принимает вектор комплексных чисел.

Потоки в GNURadio представляют собой последовательность отдельных элементов данных, которые передаются от одного блока к другому. Каждый элемент данных в потоке обрабатывается независимо от других элементов. Это делает потоки идеальными для обработки временных рядов, таких как аудиосигналы или радиосигналы.

Векторы, с другой стороны, представляют собой группу элементов данных, которые обрабатываются как единое целое. Векторы полезны, когда вам нужно обрабатывать данные в группах, например, при выполнении преобразования Фурье или других операций, которые требуют знания о множестве элементов данных одновременно. В GNURadio вы можете конвертировать потоки в векторы и наоборот с помощью специальных блоков, таких как Stream to Vector и Vector to Stream:

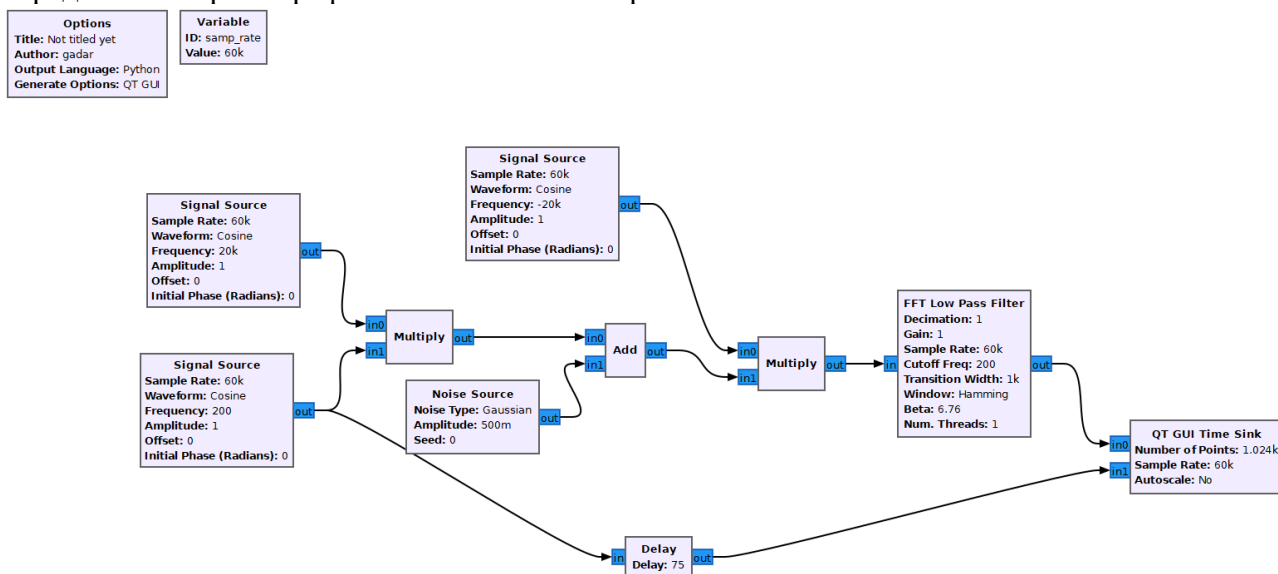


В данном случае значение Num Tups в блоках преобразования должно равняться FFT Size в блоке FFT:

Properties: Vector to Stream	
General	Advanced
IO Type	complex ▼
Num Items	1024
Vector Length	1

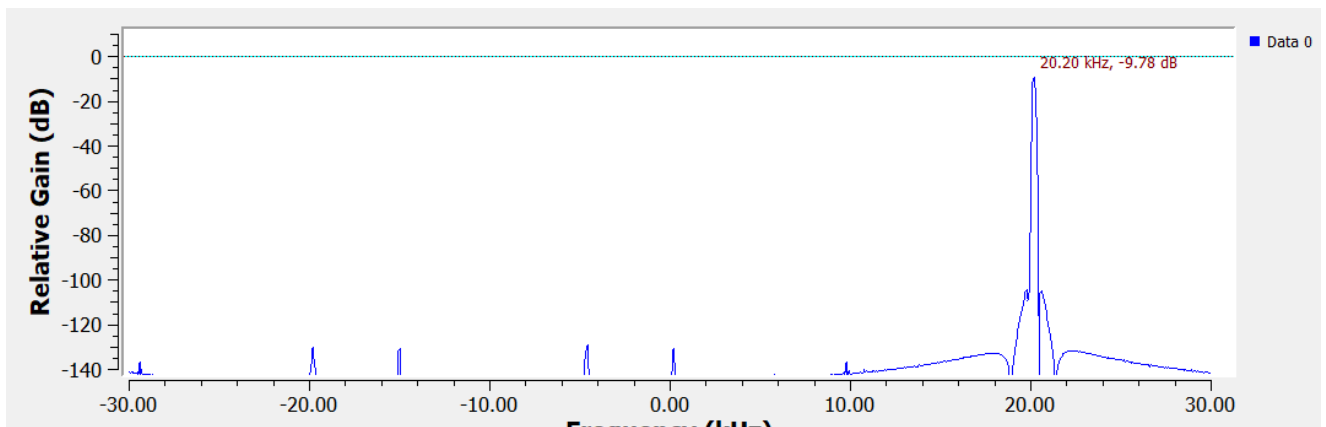
Первый граф

Разобрав основные понятия давайте, промоделируем простейшую систему прием передачи. Соберите граф как он показан на картинке:

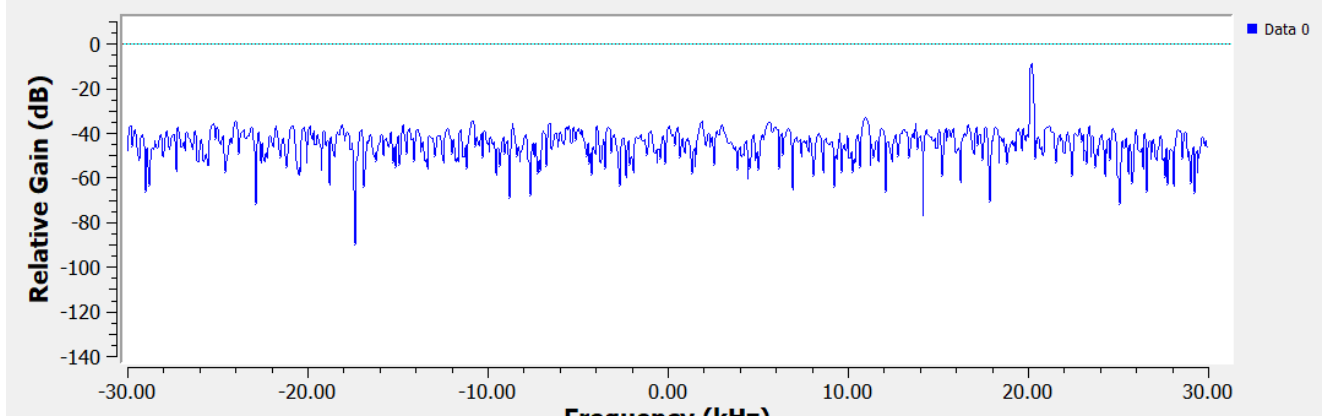


У нас есть 2 источника, один из них синтезирует несущую частоту, у нас она будет 20 кГц, второй генератор мы будем считать источником сообщений, его частота будет равна 200 Гц. Обратите внимание, что частота дискретизации была изменена на 60 кГц, для более удобного использования спектра. У источника шума амплитуда равна 0.5. У следующего источника сигнала, на который мы домножаем после шумового канала, частота равняется -20 кГц. У фильтра параметры Cutoff Freq = 200 Гц, Transition Width = 1000. В блоке Delay задержка равняется 75.

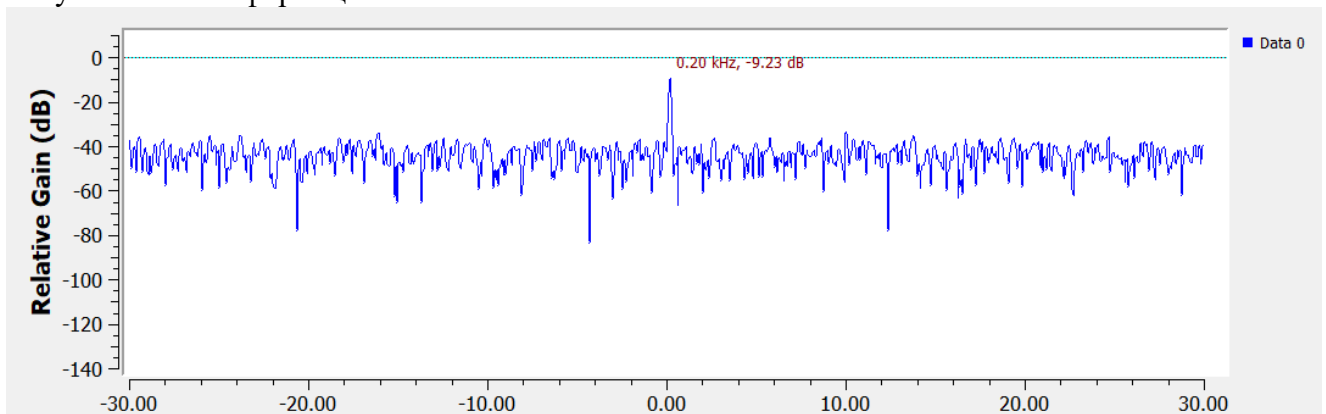
Первым шагом информационный сигнал переносится на несущую, и мы видим на спектре пик на частоте 20200 Гц. Значит всё сделано верно.



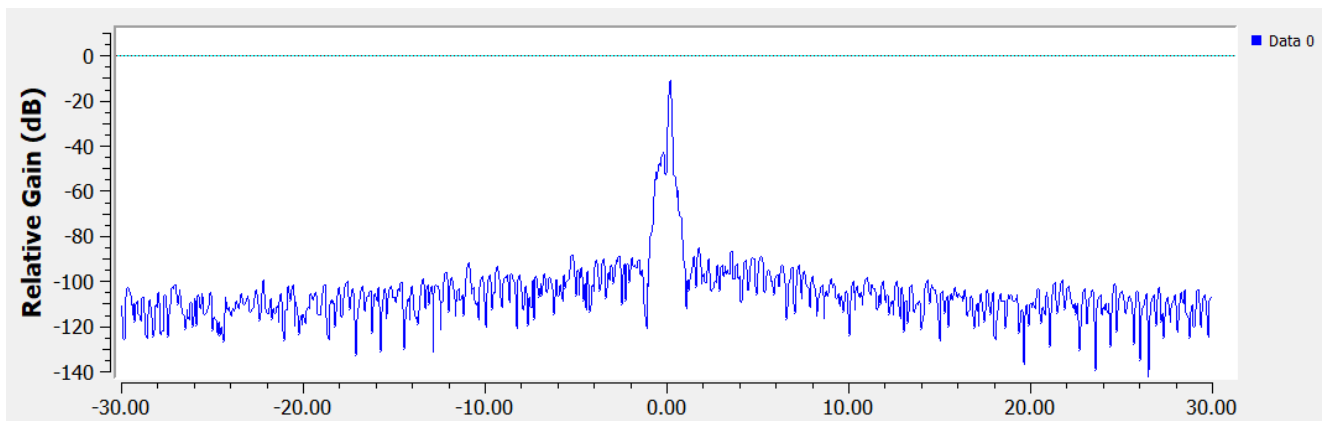
Далее мы симулируем шумовой канал, благодаря добавлению сигнала от источника шума к основному. Мы всё ещё видим сигнал за шумом



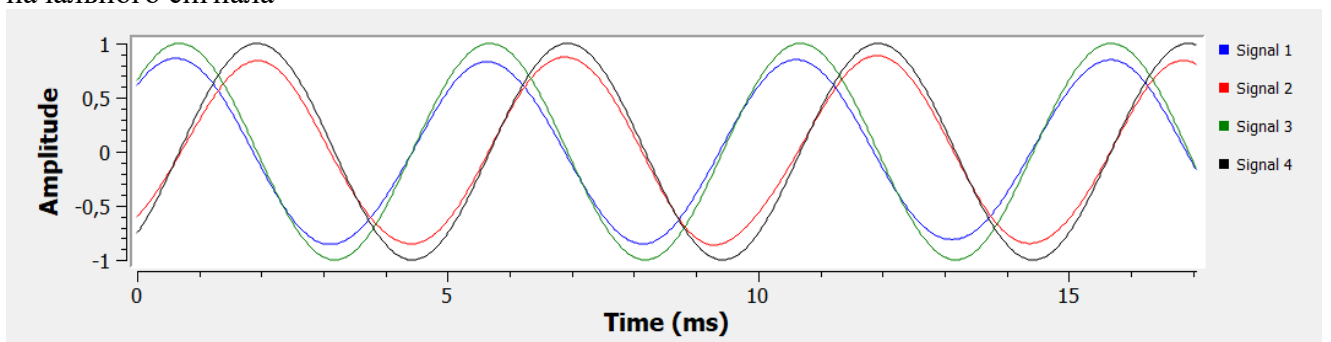
После умножаем на отрицательную частоту нашего несущего сигнала, благодаря чему получаем наш информационный сигнал.



И в конце фильтруем для избавления от шума



Далее если мы посмотрим на временной график сигнала, то он почти не отличается от начального сигнала



Создание блоков

Создавать блоки можно разными способами, но здесь будет рассмотрено создание блоков с помощью Python.

GNU Radio позволяет создавать собственные блоки на Python без необходимости создания и установки модулей Out of Tree (OOT). Вот как это можно сделать:

1. Создание блока: Добавьте в свой граф потока блок "Embedded Python Block"¹. При добавлении этого блока в ваш граф потока, предварительно заполненный код просто берет входной поток и умножает его на константу.
2. Редактирование кода блока: Чтобы изменить код Python, щелкните правой кнопкой мыши на блоке "Embedded Python Block", выберите "Properties", затем "Open in Editor" и выберите редактор¹. После завершения редактирования сохраните файл.
3. Структура блока Python: Блок Python имеет следующую структуру:

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # другие базовые классы - basic_block,
    decim_block, interp_block
    """Embedded Python Block example - a simple multiply const"""

    def __init__(self, example_param=1.0): # только аргументы по умолчанию
        здесь
        """arguments to this function show up as parameters in GRC"""
        gr.sync_block.__init__(
            self,
            name='Embedded Python Block', # будет отображаться в GRC
            in_sig=[np.complex64],
            out_sig=[np.complex64]
        )
        # если найден атрибут с тем же именем, что и параметр,
        # регистрируется обратный вызов (также работают свойства).
        self.example_param = example_param

    def work(self, input_items, output_items):
        """example: multiply with constant"""
        output_items[0][:] = input_items[0] * self.example_param
        return len(output_items[0])
```

4. В этом коде:

- `__init__` - это конструктор, который принимает параметры и инициализирует блок.
- `work` - это функция, которая обрабатывает входные и выходные данные.

5. Тестирование блока: После создания и редактирования блока вы можете добавить его в свой граф потока и протестировать его работу.

Расширение GNU Radio

GNU Radio предоставляет возможность расширять его функциональность с помощью Python и C++. Вот как это можно сделать:

- Python: Вы можете использовать Python для написания мощных приложений для обработки сигналов и радио. Это включает в себя создание собственных блоков, модификацию сгенерированных GRC Python-файлов и использование Python для создания более сложных радиоприложений.
- C++: Вы можете использовать C++ для расширения GNU Radio. Это включает в себя создание собственных блоков на C++, которые могут быть использованы для создания более сложных радиоприложений.
- gr_modtool: Инструмент gr_modtool может быть использован для создания новых модулей. Это включает в себя добавление новых блоков и модификацию существующих.
- YAML файлы: YAML файлы используются для добавления блоков в GRC3.

Дополнительная информация

Ещё больше полезной информации можно найти на сайте

<https://wiki.gnuradio.org/index.php/Tutorials> - там представлены достаточно подробные инструкции, которые помогут подробнее познакомиться с GNU Radio.